

SIMATS ENGINEERING
ASSESSMENT TOOL 1

38

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO4:

Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2, BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

1. Develop a Python program that generates a Star Topology for a given number of hosts connected to a central switch. The program should accept the number of hosts as input and display the topology either in ASCII format or a graphical representation. For each host, print the host label, the name of the central switch, and the corresponding Cat6 UTP cable length connecting the host to the switch. Verify that all hosts are connected through the central switch and explain how the star topology facilitates reliable communication in a local area network.
2. Write a Python function named `validate_segment(length_m)` to verify whether a given **UTP cable segment** satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of **100 meters**. The function should return **True** for valid cable lengths and return **False** along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least **three different test cases** and interpret the results.

3. Develop a Python program to simulate the operation of the Carrier Sense Multiple Access with Collision Detection (CSMA/CD) protocol in a star topology consisting of four hosts connected through a central switch. The program should simulate frame transmission attempts, detect transmission collisions, calculate random back-off intervals, and continue transmission after the waiting period. Record every collision event, transmission attempt, and back-off interval in a log file named `csma_log.txt`, and analyze how CSMA/CD minimizes data transmission conflicts in Ethernet networks.
4. Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.
5. Design and implement a Python class named `StarSwitch` that models the basic functionality of an Ethernet switch. The class should include methods such as `add_port(host_id)` to connect a host, `remove_port(host_id)` to disconnect a host, and `broadcast(frame)` to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

```

def main():
    num_hosts = input("Enter number of hosts: ")
    topology = StarTopology(num_hosts)
    topology.display_topology()
    topology.display_switch()
    if topology.verify_connections():
        print("All hosts are successfully connected through the central switch.")
    else:
        print("Connection verification failed.")

```

```

if __name__ == "__main__":
    main()

```

Sample Output (for 4 hosts):

Star Topology with Central Switch: Switch-1

Host	Connected To	Cable Length (m)
Host-1	Switch-1	42.17
Host-2	Switch-1	67.83
Host-3	Switch-1	12.55
Host-4	Switch-1	55.09

All hosts are successfully connected through the central switch.

Explanation:

In a star topology, every host is connected individually to a central switch rather than to each other directly. This design makes the network reliable because the failure of one cable or host affects only that single connection, while all other hosts continue to communicate normally. The central switch simplifies troubleshooting, since faults can be isolated port by port, and it enables efficient addition or removal of hosts without disturbing the rest of the network. Because each host has a dedicated Cat6 UTP link to the switch, collisions are minimized and data transmission remains fast and reliable, making the star topology the most widely used topology in modern local area networks.

Question 2

Write a Python function named `validate_segment(length, m)` to verify whether a given UTP cable segment satisfies the IEEE 802.3 Ethernet standard, which specifies a maximum cable length of 100 meters. The function should return `True` for valid cable lengths and return `False` along with an appropriate error message for invalid lengths exceeding the permissible limit. Demonstrate the correctness of the program by executing it with at least three different test cases and interpret the results.

Answer:

Program:

```
def validate_segment(length, m):  
    """  
    Validates a UTP cable segment against the IEEE 802.3  
    Ethernet standard maximum length of 100 meters.  
    Returns a tuple (bool, message).  
    """  
    MAX_LENGTH = 100  
  
    if length_m <= 0:  
        return False, "Invalid length: cable length must be greater than 0 meters."  
  
    if length_m > MAX_LENGTH:  
        return False, ("  
        Invalid: (length_m|m exceeds the IEEE 802.3 maximum "  
        "permissible length of (MAX_LENGTH|m)."  
        )  
  
    return True, ("Valid: (length_m|m is within the IEEE 802.3 permissible limit")  
  
if __name__ == "__main__":  
    test_cases = [50, 100, 120]  
  
    for length in test_cases:  
        valid, message = validate_segment(length)  
        print(f"Test case: (length)m -> Valid: {valid} | {message}")
```

Test Case Output:

Test case: 50m -> Valid: True | Valid: 50m is within the IEEE 802.3 permissible limit.
Test case: 100m -> Valid: True | Valid: 100m is within the IEEE 802.3 permissible limit.
Test case: 120m -> Valid: False | Invalid: 120m exceeds the IEEE 802.3 maximum permissible length of 100m

Interpretation:

The first test case (50m) is well within the standard limit and is correctly validated as `True`. The second test case (100m) represents the exact boundary condition specified by IEEE 802.3, and the function correctly treats it as valid since the standard allows lengths up to and including 100

minimize, and troubleshoot the rural network remotely without expensive site visits.

To ensure secure communication with headquarters, a Site-to-Site IPsec VPN (Virtual Private Network) utilizing AES-256 encryption must be configured on the SD-WAN router. This creates a secure, encrypted tunnel over the public internet, protecting sensitive corporate data from interception. A Stateful Packet Inspection (SPI) Firewall with Intrusion Prevention System (IPS) capabilities should also be integrated at the edge to block unauthorized incoming traffic and mitigate advanced cyber threats.

This recommended architecture strictly adheres to the provided constraints. By combining satellite and LTE, it overcomes the bandwidth limitations of traditional rural networks while maintaining high reliability through automated failover. The SD-WAN router optimizes the limited budget by avoiding expensive leased lines (like MPLS) and maximizing the efficiency of affordable wireless links. Finally, this setup offers excellent future scalability; as the branch grows, additional LTE modems or higher-tier satellite plans can be seamlessly integrated into the existing SD-WAN infrastructure without requiring a complete hardware overhaul.

Intrusion Detection System (NIDS) should be deployed via port mirroring (SPAN) on the core switch. This allows the NIDS to passively analyze traffic for malicious anomalies, such as ransomware propagation or unauthorized data exfiltration, without introducing any inline network delay.

Secure access must be regulated through strict Identity and Access Management (IAM) protocols, including Multi-Factor Authentication (MFA) and Role-Based Access Control (RBAC). When employees attempt to access sensitive financial records, they must verify their identity using a hardware security key or authenticator app in addition to their password. RBAC ensures that employees only have access to the specific data necessary for their job roles, strictly adhering to the principle of least privilege. Furthermore, all internal data in transit must be encrypted using TLS 1.3, and data at rest utilizing AES-256 encryption, complemented by a Data Loss Prevention (DLP) system to monitor and prevent sensitive customer information from leaving the corporate network.

This architecture perfectly addresses the financial institution's constraints. It delivers maximum security and regulatory compliance by isolating sensitive data, enforcing encryption, and requiring MFA. High performance is maintained because the NIDS operates completely out-of-band, and the NGFW is hardware-accelerated for internal routing. Finally, budget constraints are met by utilizing logical VLAN segmentation and open-source NIDS rather than purchasing disparate hardware appliances for every department.

Q3: A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each branch based on overall link quality rather than just the shortest distance. Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

Answer:

Algorithm Adaptive_Optimal_Path_Selector

Inputs: hq_to_branches_paths = [Path1, Path2, Path3, ...]

BEGIN

WHILE System_Active DO

 best_communication_path = NULL

 highest_quality_score = -9999

 FOR EACH path IN hq_to_branches_paths DO

 // Periodically collect dynamic link metrics

 bandwidth = monitor_bandwidth(path)

 delay = monitor_delay(path)

 utilization = monitor_utilization(path)

 // Store collected values in historical lists

 path.bandwidth_history.add(bandwidth)

 path.delay_history.add(delay)

 path.utilization_history.add(utilization)

 // Detect failed or overloaded links

connected to a central Layer 3 switch or router, which acts as the core layer and enables communication between different departments through inter-VLAN routing. Virtual LANs (VLANs) are configured to logically separate each department's network, improving security and reducing broadcast traffic. A router is also used to connect the internal network to external networks such as the internet, often with firewall functionality for security. Redundant links and backup paths can be included to ensure reliability, while proper IP addressing and subnetting are applied for efficient network management. This hierarchical design improves performance, enhances security, and allows easy expansion as the company grows.

Q2: An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.

Answer:

Algorithm Bandwidth_Management_System

Inputs:

network_links_list = [Link1, Link2, Link3, ...]

monitoring_interval = 5 seconds

BEGIN

// Start continuous loop to monitor all links

WHILE System_is_Running DO

FOR EACH link IN network_links_list DO

// Collect current bandwidth usage

current_utilization = get_link_utilization(link)

// Store the values in a list for tracking

link_usage_history[link.id].append(current_utilization)

// Check if bandwidth utilization exceeds threshold

IF current_utilization > 80 THEN

PRINT "ALERT: Link " + link.id + " Overloaded (" + current_utilization +

"%)!"

// Identify an available backup link

backup_link = find_available_backup_link()

// Recommend and execute traffic switch

PRINT "Action Recommended: Switching traffic to " + backup_link.id

switch_traffic(link, backup_link)

ELSE